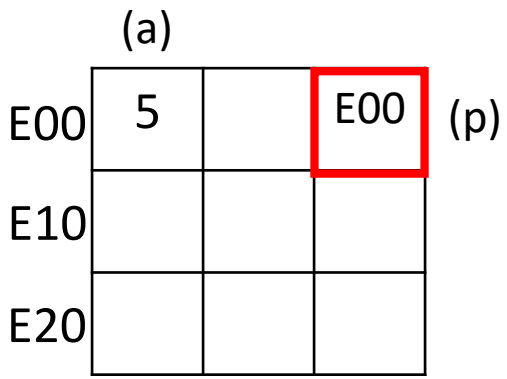
O que são ponteiros?

- **Um ponteiro é um endereço de memória**. Seu valor indica onde uma variável está armazenada, não o que está armazenado. Um ponteiro proporciona um modo de acesso a uma variável sem referencia-la diretamente.

Memória



No exemplo ao lado, “a” é uma variável e seu valor é 5 (a = 5). E “p” é um ponteiro que **armazena o endereço** de memória da variável “a” (E00).

Obs.: As posições E00, E10, ... , E22 significam os endereços da memória

O que são ponteiros?

- Para dominar a linguagem C, é essencial dominar ponteiros. Os ponteiros são usados em situações em que o uso de uma variável é difícil ou indesejado.
- Algumas razões para o uso de ponteiros:
  - manipular elementos de matrizes;
  - receber argumentos em funções que necessitem modificar o argumento original
  - passar cadeias de caracteres (strings) de uma função para outra; usá-los no lugar de matrizes
  - criar estruturas de dados complexas, como listas encadeadas e árvores binárias, onde um item deve conter referências a outro;
  - alocar e desalocar memória do sistema

Endereços de Memória

- A memória de seu computador é dividida em bytes, e estes bytes são numerados de 0 até o limite de sua máquina. Estes números são chamados endereços de bytes. **Um endereço é a referência que o computador usa para localizar variáveis**.
- Toda variável ocupa certa localização na memória, e seu endereço é o do primeiro byte ocupado por ela.

Endereço na memória	Variável na memória
1000	13
1001	10
1002	A
...	...
N	

Endereços de Memória

- Nossos programas, quando carregados para a memória, ocupam uma certa parte dela. Dessa forma, **toda variável e toda função de nosso programa começam em um endereço particular**, e este endereço é chamado **endereço da variável ou função**

O operador de endereços “&”

- Para conhecermos o endereço ocupado por uma variável usamos o operador de endereços “&”.

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int i, j, k;
    printf("%p\n%p\n%p\n",&i,&j,&k);

    return 0;
}
```

Resultado:

```
0061ff04
0061ff08
0061ff0c
```

Obs.: “%p” é o formato específico para imprimir na tela o endereço de uma variável.

Declarando Ponteiros

- Um ponteiro, como qualquer variável, deve ter um tipo, que é o tipo da variável para a qual ele aponta. Para declarar um ponteiro, especificamos o tipo da variável para a qual ele aponta e seu nome precedido por asterisco:

```
int ponteiro;          // declara uma variável comum do tipo inteiro
int *ponteiro;         // declara um ponteiro para um inteiro
```

Declarando Ponteiros

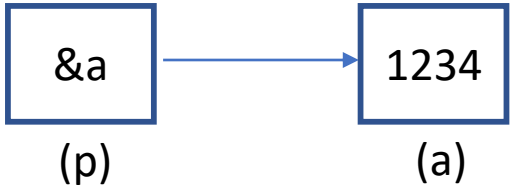
- Tome cuidado ao declarar vários ponteiros em uma linha, pois o asterisco deve vir antes de cada nome de variável. Note os três exemplos:

```
int p, q, r;          // estamos a declarar três variáveis comuns
int *p, q, r;         // cuidado! apenas p será um ponteiro!
int *p, *q, *r;       // agora sim temos três ponteiros
```

Declarando Ponteiros

- Para atribuir um valor ao ponteiro, usamos apenas seu nome de variável. Esse valor deve ser um endereço de memória, portanto obtido com o operador &:

```
int a = 1234;
int *p;
p = &a;
```



- Também podemos inicializar um ponteiro:

```
int *p = &a;
```

Declarando Ponteiros

- Como o ponteiro contém um endereço, podemos também atribuir um valor à variável guardada nesse endereço, ou seja, à variável apontada pelo ponteiro.
- Para isso, usamos o operador * (asterisco), que significa "o valor apontado por"

```
int main()
{
    int a = 10;
    int *p;
    p = &a;
    *p = 5;
    printf ("%d\t%d\t%p\n", a, *p, p);

    return 0;
}
```

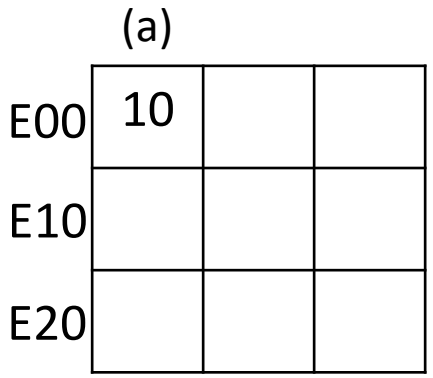
Declarando Ponteiros

- Como o ponteiro contém um endereço, podemos também atribuir um valor à variável guardada nesse endereço, ou seja, à variável apontada pelo ponteiro.
- Para isso, usamos o operador * (asterisco), que significa "o valor apontado por"

```
int main()
{
    int a = 10;
    int *p;
    p = &a;
    *p = 5;
    printf ("%d\t%d\t%p\n", a, *p, p);

    return 0;
}
```

Memória



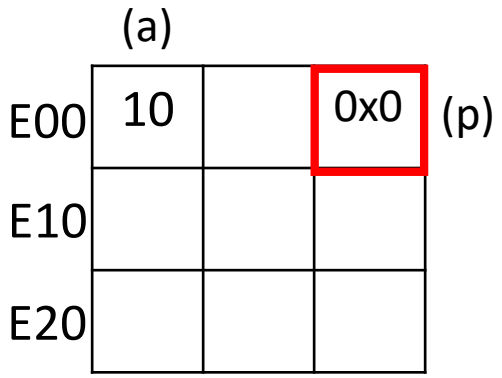
Declarando Ponteiros

- Como o ponteiro contém um endereço, podemos também atribuir um valor à variável guardada nesse endereço, ou seja, à variável apontada pelo ponteiro.
- Para isso, usamos o operador * (asterisco), que significa "o valor apontado por"

```
int main()
{
    int a = 10;
    int *p;
    p = &a;
    *p = 5;
    printf ("%d\t%d\t%p\n", a, *p, p);

    return 0;
}
```

Memória

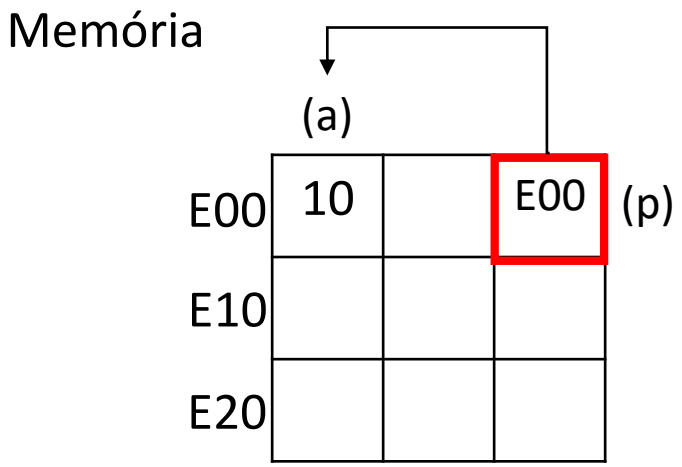


Declarando Ponteiros

- Como o ponteiro contém um endereço, podemos também atribuir um valor à variável guardada nesse endereço, ou seja, à variável apontada pelo ponteiro.
- Para isso, usamos o operador * (asterisco), que significa "o valor apontado por"

```
int main()
{
    int a = 10;
    int *p;
    p = &a;
    *p = 5;
    printf ("%d\t%d\t%p\n", a, *p, p);

    return 0;
}
```

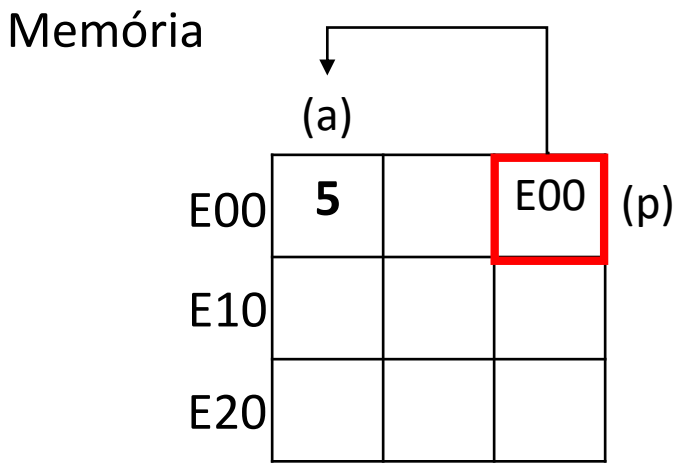


Declarando Ponteiros

- Como o ponteiro contém um endereço, podemos também atribuir um valor à variável guardada nesse endereço, ou seja, à variável apontada pelo ponteiro.
- Para isso, usamos o operador * (asterisco), que significa "o valor apontado por"

```
int main()
{
    int a = 10;
    int *p;
    p = &a;
    *p = 5;
    printf ("%d\t%d\t%p\n", a, *p, p);

    return 0;
}
```



Declarando Ponteiros

- Como o ponteiro contém um endereço, podemos também atribuir um valor à variável guardada nesse endereço, ou seja, à variável apontada pelo ponteiro.
- Para isso, usamos o operador * (asterisco), que significa "o valor apontado por"

```
int main()
{
    int a = 10;
    int *p;
    p = &a;
    *p = 5;
    printf ("%d\t%d\t%p\n", a, *p, p);

    return 0;
}
```

"%d": imprime o tipo "int", ou seja, o valor apontado por "*p"

"%p": imprime o endereço de memória do ponteiro "p"

"\t": tabular

Saída:

```
5        5        E00
a,       *p,      p
```

Declarando Ponteiros

- Como o ponteiro contém um endereço, podemos também atribuir um valor à variável guardada nesse endereço, ou seja, à variável apontada pelo ponteiro.
- Para isso, usamos o operador * (asterisco), que significa "o valor apontado por"

```
int main()
{
    int a = 10;
    int *p;
    p = &a;
    *p = 5;
    printf ("%d\t%d\t%p\n", a, *p, p);

    return 0;
}
```

O ponteiro “p” recebe apenas endereços de memória de variáveis, que são atribuídos utilizando o “&” (dizemos que um ponteiro recebe a referência da variável na memória)

“*p” recebe o valores do tipo do ponteiro. Estes valores são atribuídos à variável referenciada pelo ponteiro.

Ponteiro e NULL

- Uma falha de segmentação ou em inglês *"segmentation fault"* ocorre quando um programa tenta acessar um endereço na memória que está reservado ou que não existe.
- Aqui o ponteiro contem NULL, definido com o endereço (0x00000000) que causa uma falha de segmentação.
- É sempre bom inicializarmos os ponteiros, pois senão eles podem vir com lixo e você se esquecer, posteriormente, de inicializar.

```
int *p = NULL;
```

Uma boa prática é apontar os ponteiros para a primeira posição de memória, ou seja, NULL

Operação com Ponteiros

- Suponhamos dois ponteiros inicializados p1 e p2. Podemos fazer dois tipos de atribuição entre eles:
  - Esse primeiro exemplo fará com que p1 aponte para o mesmo lugar que p2. Ou seja, usar p1 será equivalente a usar p2 após essa atribuição

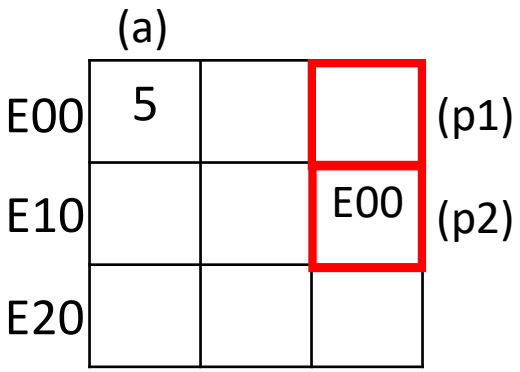
```
p1 = p2;
```

  - Nesse segundo caso, igualamos os valores apontados pelos dois ponteiros: alteraremos o valor apontado por p1 para o valor apontado por p2.

```
*p1 = *p2;
```

```
int a = 5;
int *p1, *p2;
p2 = &a;
```

Memória



Operação com Ponteiros

- Suponhamos dois ponteiros inicializados p1 e p2. Podemos fazer dois tipos de atribuição entre eles:
  - Esse primeiro exemplo fará com que p1 aponte para o mesmo lugar que p2. Ou seja, usar p1 será equivalente a usar p2 após essa atribuição

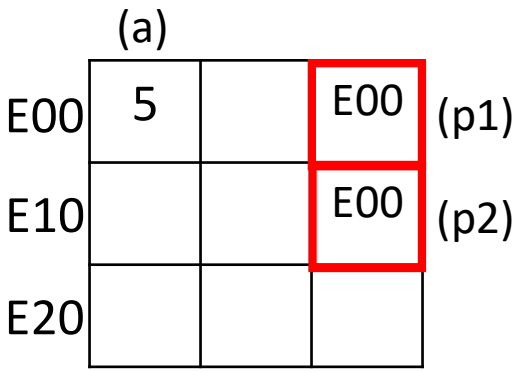
```
p1 = p2;
```

  - Nesse segundo caso, igualamos os valores apontados pelos dois ponteiros: alteraremos o valor apontado por p1 para o valor apontado por p2.

```
*p1 = *p2;
```

```
int a = 5;
int *p1, *p2;
p2 = &a;
```

Memória



Operação com Ponteiros

- Suponhamos dois ponteiros inicializados p1 e p2. Podemos fazer dois tipos de atribuição entre eles:
  - Esse primeiro exemplo fará com que p1 aponte para o mesmo lugar que p2. Ou seja, usar p1 será equivalente a usar p2 após essa atribuição

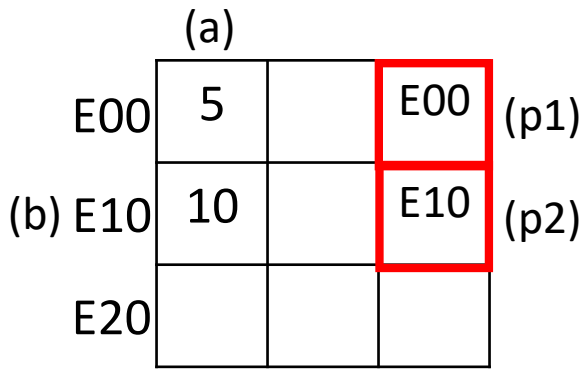
```
p1 = p2;
```

  - Nesse segundo caso, igualamos os valores apontados pelos dois ponteiros: alteraremos o valor apontado por p1 para o valor apontado por p2.

```
*p1 = *p2;
```

```
int a = 5;
int b = 10;
int *p1, *p2;
p1 = &a;
p2 = &b;
```

Memória



Operação com Ponteiros

- Suponhamos dois ponteiros inicializados p1 e p2. Podemos fazer dois tipos de atribuição entre eles:
  - Esse primeiro exemplo fará com que p1 aponte para o mesmo lugar que p2. Ou seja, usar p1 será equivalente a usar p2 após essa atribuição

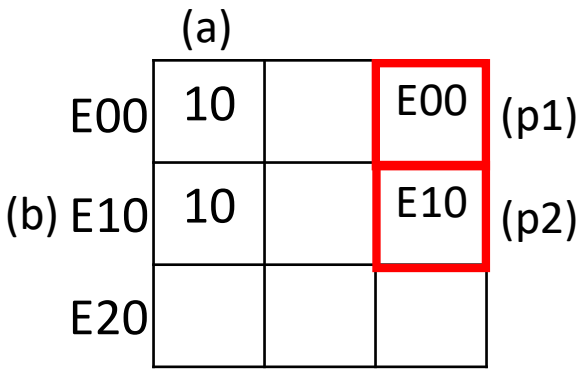
```
p1 = p2;
```

  - Nesse segundo caso, igualamos os valores apontados pelos dois ponteiros: alteraremos o valor apontado por p1 para o valor apontado por p2.

```
*p1 = *p2;
```

```
int a = 5;
int b = 10;
int *p1, *p2;
p1 = &a;
p2 = &b;
```

Memória



Ponteiros e vetores

- Em C, os elementos de um vetor são sempre guardados sequencialmente, a uma distância fixa um do outro.
- Com isso, é possível facilmente passar de um elemento a outro, percorrendo sempre uma mesma distância para frente ou para trás na memória.
- Dessa maneira, podemos usar ponteiros e a aritmética de ponteiros para percorrer vetores. Na verdade, **vetores são ponteiros** - um uso particular dos ponteiros.

Ponteiros e vetores

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int i;
    int vetor[3] = {4, 7, 1};
    int *ptr = vetor;
    printf("%p\n", vetor);
    printf("%p\n", ptr);

    for (i = 0; i < 3; i++) {
        printf("O endereco do indice %d do vetor eh %p\n", i, &ptr[i]);
        printf("O valor do indice %d do vetor eh %d\n", i, ptr[i]);
    }

    return 0;
}
```

Saída:

```
0061ff04
0061ff04
O endereco do indice 0 do vetor eh 0061ff04
O valor do indice 0 do vetor eh 4
O endereco do indice 1 do vetor eh 0061ff08
O valor do indice 1 do vetor eh 7
O endereco do indice 2 do vetor eh 0061ff0c
O valor do indice 2 do vetor eh 1
```

Passando Argumentos por referência

- Qual será o resultado do programa abaixo?

```
void trocar(int num1, int num2) {
    int temp;
    temp = num1;
    num1 = num2;
    num2 = temp;
}
int main() {
    int a, b;
    a = 5;
    b = 10;
    printf ("\n\nEles valem %d, %d\n", a, b);

    trocar (a, b);

    printf ("\n\nEles agora valem %d, %d\n", a, b);

    return 0;
}
```

Passando Argumentos por referência

- Qual será o resultado do programa abaixo?

```
void trocar(int num1, int num2) {
    int temp;
    temp = num1;
    num1 = num2;
    num2 = temp;
}
int main() {
    int a, b;
    a = 5;
    b = 10;
    printf ("\n\nEles valem %d, %d\n", a, b);

    trocar (a, b);

    printf ("\n\nEles agora valem %d, %d\n", a, b);

    return 0;
}
```

Saída:

```
Eles valem 5, 10
Eles agora valem 5, 10
```

Passando Argumentos por referência

- Qual será o resultado do programa abaixo?

```
void trocar(int num1, int num2) {
    int temp;
    temp = num1;
    num1 = num2;
    num2 = temp;
}
int main() {
    int a, b;
    a = 5;
    b = 10;
    printf ("\n\nEles valem %d, %d\n", a, b);

    trocar (a, b);

    printf ("\n\nEles agora valem %d, %d\n", a, b);

    return 0;
}
```

Memória

(a) E00	5		
(b) E10	10		
E20			

Passando Argumentos por referência

- Qual será o resultado do programa abaixo?

```
void trocar(int num1, int num2) {
    int temp;
    temp = num1;
    num1 = num2;
    num2 = temp;
}
int main() {
    int a, b;
    a = 5;
    b = 10;
    printf ("\n\nEles valem %d, %d\n", a, b);

    trocar (a, b);

    printf ("\n\nEles agora valem %d, %d\n", a, b);

    return 0;
}
```

Memória

(a) E00	5		5	(num1)
(b) E10	10		10	(num2)
E20				

Passando Argumentos por referência

- Qual será o resultado do programa abaixo?

```
void trocar(int num1, int num2) {
    int temp;
    temp = num1;
    num1 = num2;
    num2 = temp;
}
int main() {
    int a, b;
    a = 5;
    b = 10;
    printf ("\n\nEles valem %d, %d\n", a, b);

    trocar (a, b);

    printf ("\n\nEles agora valem %d, %d\n", a, b);

    return 0;
}
```

Memória

(a) E00	5		10	(num1)
(b) E10	10		5	(num2)
E20			5	(temp)

Passando Argumentos por referência

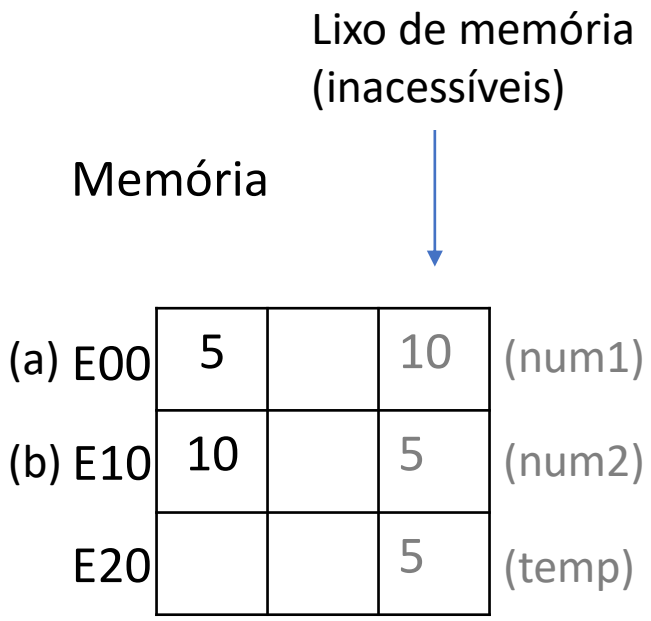
- Passagem de parâmetros por valor. A função não altera os valores.

```
void trocar(int num1, int num2) {
    int temp;
    temp = num1;
    num1 = num2;
    num2 = temp;
}
int main() {
    int a, b;
    a = 5;
    b = 10;
    printf ("\n\nEles valem %d, %d\n", a, b);

    trocar (a, b);

    printf ("\n\nEles agora valem %d, %d\n", a, b);

    return 0;
}
```



Passando Argumentos por referência

- A troca não é realizada devido ao escopo das variáveis serem locais na função **trocar**!

- Neste caso podemos usar a passagem do argumento como referência para que a troca seja feita na função e continue efetiva dentro da função principal.

Passando Argumentos por referência

- Qual será o resultado do programa abaixo?

```
void trocar(int *num1, int *num2) {
    int temp;
    temp = *num1;
    *num1 = *num2;
    *num2 = temp;
}
int main() {
    int a, b;
    a = 5;
    b = 10;
    printf ("\n\nEles valem %d, %d\n", a, b);

    trocar (&a, &b);

    printf ("\n\nEles agora valem %d, %d\n", a, b);

    return 0;
}
```

Passando Argumentos por referência

- Qual será o resultado do programa abaixo?

```
void trocar(int *num1, int *num2) {
    int temp;
    temp = *num1;
    *num1 = *num2;
    *num2 = temp;
}
int main() {
    int a, b;
    a = 5;
    b = 10;
    printf ("\n\nEles valem %d, %d\n", a, b);

    trocar (&a, &b);

    printf ("\n\nEles agora valem %d, %d\n", a, b);

    return 0;
}
```

Memória

(a) E00	5		
(b) E10	10		
E20			

Passando Argumentos por referência

- Qual será o resultado do programa abaixo?

```
void trocar(int *num1, int *num2) {
    int temp;
    temp = *num1;
    *num1 = *num2;
    *num2 = temp;
}
int main() {
    int a, b;
    a = 5;
    b = 10;
    printf ("\n\nEles valem %d, %d\n", a, b);

    trocar (&a, &b);

    printf ("\n\nEles agora valem %d, %d\n", a, b);

    return 0;
}
```

Equivalente a:

```
int *num1;
num1 = &a
```

Memória

(a) E00	5		E00	(num1)
(b) E10	10		E10	(num2)
E20				

Passando Argumentos por referência

- Qual será o resultado do programa abaixo?

```
void trocar(int *num1, int *num2) {
    int temp;
    temp = *num1;
    *num1 = *num2;
    *num2 = temp;
}
int main() {
    int a, b;
    a = 5;
    b = 10;
    printf ("\n\nEles valem %d, %d\n", a, b);

    trocar (&a, &b);

    printf ("\n\nEles agora valem %d, %d\n", a, b);

    return 0;
}
```

Memória

(a) E00	5		E00	(num1)
(b) E10	10		E10	(num2)
E20			5	(temp)

Passando Argumentos por referência

- Qual será o resultado do programa abaixo?

```
void trocar(int *num1, int *num2) {
    int temp;
    temp = *num1;
    *num1 = *num2;
    *num2 = temp;
}
int main() {
    int a, b;
    a = 5;
    b = 10;
    printf ("\n\nEles valem %d, %d\n", a, b);

    trocar (&a, &b);

    printf ("\n\nEles agora valem %d, %d\n", a, b);

    return 0;
}
```

Memória

(a) E00	10		E00	(num1)
(b) E10	5		E10	(num2)
E20			5	(temp)

Passando Argumentos por referência

- Qual será o resultado do programa abaixo?

```
void trocar(int *num1, int *num2) {
    int temp;
    temp = *num1;
    *num1 = *num2;
    *num2 = temp;
}
int main() {
    int a, b;
    a = 5;
    b = 10;
    printf ("\n\nEles valem %d, %d\n", a, b);

    trocar (&a, &b);

    printf ("\n\nEles agora valem %d, %d\n", a, b);

    return 0;
}
```

Saída:

```
Eles valem 5, 10
Eles agora valem 10, 5
```

Memória

(a) E00	10		E00	(num1)
(b) E10	5		E10	(num2)
E20			5	(temp)

Lixo de memória (inacessíveis)

Passando Argumentos por referência

- E agora (com vetor), qual será o resultado do programa abaixo?

```
void trocar(int num[]) {
    int temp;
    temp = num[0];
    num[0] = num[1];
    num[1] = temp;
}
int main(){
    int numeros[2];
    numeros[0] = 5;
    numeros[1] = 10;

    printf ("Eles valem %d, %d\n",numeros[0],numeros[1]);
    trocar (numeros);
    printf ("Eles agora valem %d, %d\n",numeros[0],numeros[1]);

    return 0;
}
```

Passando Argumentos por referência

- E agora (com vetor), qual será o resultado do programa abaixo?

```
void trocar(int num[]) {
    int temp;
    temp = num[0];
    num[0] = num[1];
    num[1] = temp;
}
int main(){
    int numeros[2];
    numeros[0] = 5;
    numeros[1] = 10;

    printf ("Eles valem %d, %d\n",numeros[0],numeros[1]);
    trocar (numeros);
    printf ("Eles agora valem %d, %d\n",numeros[0],numeros[1]);

    return 0;
}
```

Ao passar vetor por parâmetros a uma função o endereço do vetor é atribuído à função. Toda alteração no vetor será refletida no programa principal.

Saída:

```
Eles valem 5, 10
Eles agora valem 10, 5
```

Passando Argumentos por referência

- O mesmo pode ser feito utilizando ponteiros

```
void trocar(int *num) {
    int temp;
    temp = num[0];
    num[0] = num[1];
    num[1] = temp;
}
int main(){
    int numeros[2];
    numeros[0] = 5;
    numeros[1] = 10;

    printf ("Eles valem %d, %d\n",numeros[0],numeros[1]);
    trocar (numeros);
    printf ("Eles agora valem %d, %d\n",numeros[0],numeros[1]);

    return 0;
}
```

Saída:

```
Eles valem 5, 10
Eles agora valem 10, 5
```

Alocação Dinâmica

Alocação Dinâmica

- É muito comum necessitar reservar um certo espaço na memória para poder guardar dados mais tarde.
  - Por exemplo: reservar um espaço de 1000 bytes para guardar uma cadeia de caracteres (string) para entrada pelo teclado, declarando um vetor de 1000 caracteres.
- Como reservar um espaço que só é conhecido no tempo de execução do programa?
- E se o espaço fosse muito grande, de modo que declarar vetores de tal tamanho não fosse possível (ou aumentasse o tamanho da memória utilizada)?

Alocação Dinâmica

- A **alocação dinâmica de memória**, que como o nome sugere, é uma maneira de alocar memória durante a execução do programa.
- A alocação dinâmica é gerenciada pelas funções **malloc** e **free**, da biblioteca <**stdlib.h**>.
  - **malloc** é responsável pela alocação de um pedaço de memória, e
  - **free** é responsável por liberar esse pedaço de memória.

malloc e free

- A função **malloc** (abreviatura de *memory allocation*) aloca um bloco de bytes consecutivos na memória do computador e devolve o endereço desse bloco. O número de bytes é especificado no argumento da função.
- A função **free** é responsável por liberar a memória alocada.

```
void *malloc (unsigned int num);
void free (void * ptr);
```

malloc e free

- Para alocar um espaço na memória, precisamos fornecer à função **malloc** o número de bytes desejados.
- Ela aloca na memória e retorna um **ponteiro** para o primeiro byte alocado.
  - Se não houver memória suficiente para alocar a memória requisitada a função **malloc()** retorna um ponteiro nulo(NULL).

malloc e free

Número de bytes (quantidade) que será alocado (tamanho do tipo inteiro)

```
#include <stdio.h>
#include <stdlib.h>

int main(){
    int a;                                  //alocação estática
    int *p = (int*)malloc(sizeof(int));     //alocação dinâmica

    if ( p == NULL ){
        printf ("Nao foi possivel alocar memoria!\n");
        exit(1);
    }

    a = 10;
    *p = 5;

    printf("Numeros: %d, %d\n\n",a,*p);
    free(p);

    return 0;
}
```

Saída:

```
Numeros: 10, 5
```

calloc

- A função **calloc()** também serve para alocar memória, mas possui um protótipo um pouco diferente:

```
void *calloc(size_t nelem, size_t elsize);
```

- A função calloc reserva um bloco com o tamanho (nelem x elsize) octetos consecutivos, isto é, **aloca memória suficiente para um vetor de "num" objetos de tamanho "size"**.
- Se não houver memória suficiente para alocar a memória requisitada a função calloc() retorna um ponteiro nulo (NULL).

calloc

```c
#include <stdio.h>
#include <stdlib.h>

int main(){
     int i;
     int n=10;

     int *p = (int *)calloc(n, sizeof(int));

     if (p == NULL) {
          printf ("** Erro: Memoria Insuficiente **");
          exit(0);
     }

     for (i=0; i<n; i++)
          p[i] = i*i;

     for (i=0; i<n; i++)
          printf("%d ",p[i]);

     free(p);
     return 0;
}
```

Alocação memória de “n” posições de inteiros (tamanho “n * sizeof(int)”)

Saída:

```
0 1 4 9 16 25 36 49 64 81
```

calloc

```
#include <stdio.h>
#include <stdlib.h>

int main(){
    int i;
    int n;
    scanf("%d", &n);

    int *p = (int *)calloc(n, sizeof(int));

    if (p == NULL) {
        printf ("** Erro: Memoria Insuficiente **");
        exit(0);
    }

    for (i=0; i<n; i++)
        p[i] = i*i;

    for (i=0; i<n; i++)
        printf("%d ",p[i]);

    free(p);
    return 0;
}
```

Tamanho “n” é definido em tempo de execução.

Alocação dinâmica de vetores

```
float *Alocar_vetor_real (int n){
    if (n < 1){
        printf ("** Erro: Parametro invalido **\n");
        return (NULL);
    }

    float * v = (float*)calloc (n, sizeof(float));

    if (v == NULL){
        printf ("** Erro: Memoria Insuficiente **");
        return (NULL);
    }

    return (v);
}
void Liberar_vetor_real (float *v)
{
    if (v != NULL)
        free(v);
}
```

Alocação de vetor em tempo de execução.

Retorna o endereço do vetor “v” instanciado .

Libera o espaço de memória alocado para o vetor.

Alocação dinâmica de vetores

```
int main (void) {
    int i;
    int TAM;

    do{
        printf("Digite o tamanho de TAM (> 0): ");
        scanf("%d", &TAM);
    } while(TAM <=0);

    float *vetor_real = Alocar_vetor_real(TAM);

    for (i=0; i<TAM; i++)
        vetor_real[i] = i;

    for (i=0; i<TAM; i++)
        printf("\n%.2f",vetor_real[i]);

    Liberar_vetor_real (vetor_real);

    return 0;
}
```

Observe que o tamanho do vetor será informado pelo usuário em tempo de execução

A função `Alocar_vetor_real` retorna um vetor (endereço do vetor na memória) instanciado para a função principal.

Chamada da função `Liberar_vetor_real` que libera o espaço de memória alocado para o vetor. Observe que é passado como argumento o endereço de memória do vetor.

Alocação Dinâmica de Matrizes

```c
int **Alocar_matriz (int m, int n) {
    int i;
    if (m < 1 || n < 1) {
        printf ("** Erro: Parametro invalido **\n");
        return (NULL);
    }

    int **matriz = (int **)calloc (m, sizeof(int*));

    if (matriz == NULL){
        printf ("** Erro: Memoria Insuficiente **");
        return (NULL);
    }

    for (i = 0; i < m; i++ ) {
        matriz[i] = (int*)calloc (n, sizeof(int));

        if (matriz[i] == NULL){
            printf ("** Erro: Memoria Insuficiente **");
            return (NULL);
        }
    }

    return (matriz);
}
```

Alocação de um array bidimensional de tamanho “m*int” em uma das dimensões. Observe o uso de “**” para denotar duas dimensões.

Em uma alocação de matriz em C, para cada uma de suas “m” posições na memória, é necessário alocar as “n” posições.

Retorna o endereço da matriz instanciada para a função principal .

Alocação Dinâmica de Matrizes

```
void Liberar_matriz (int m, int n, int **mat) {
    int i;
    if (mat != NULL) {
        if (m < 1 || n < 1)
            printf ("** Erro: Parametro invalido **\n");
        else {
            for (i=0; i<m; i++)
                free (mat[i]);
            free (mat);
        }
    }
}
```

Alocação Dinâmica de Matrizes

```
int main (){
     int i, j;
     int TAM_M, TAM_N;

     printf("Digite o tamanho de TAM_M (> 0): ");
     scanf("%d", &TAM_M);
     printf("Digite o tamanho de TAM_N (> 0): ");
     scanf("%d", &TAM_N);

     int **MATRIZ = Alocar_matriz(TAM_M,TAM_N);

     for (i=0; i<TAM_M; i++)
          for (j=0; j<TAM_N; j++)
               MATRIZ[i][j] = i + j;

     for (i=0; i<TAM_M; i++){
          for (j=0; j<TAM_N; j++)
               printf("%d ",MATRIZ[i][j]);
          printf("\n");
     }

     Liberar_matriz(TAM_M,TAM_N,MATRIZ);

     return 0;
}
```

Recebe o endereço da Matriz alocada dinamicamente

Chamada da função que libera espaço de memória alocada para a matriz

Alocação Dinâmica de Estruturas

```
typedef struct{
     int dia, mes, ano;
} DATA;

int main(){
     int i; int TAM=0;

     printf("\nDigite o valor de TAM: ");
     scanf("%d", &TAM);

     DATA *datas = (DATA*)malloc(sizeof(DATA) * TAM);
     if (datas == NULL){
          printf ("** Erro: Memoria Insuficiente **"); exit(0);
     }

     for(i=0;i<TAM;i++){
          datas[i].dia = rand()%28 + 1;
          datas[i].mes = rand()%12 + 1;
          datas[i].ano = 2000 + rand()%18;
     }
     for(i=0;i<TAM;i++)
          printf("Dia: %d/%d/%d\n", datas[i].dia, datas[i].mes, datas[i].ano);

     free(datas);
     return 0;
}
```

Alocação dinâmica do ponteiro “**datas**” do tipo abstrato **DATA**. Observe que o espaço alocado em memória é o tamanho do tipo DATA multiplicado pela quantidade de “datas” que serão armazenadas pelo usuário.

Exercícios

- Faça um programa que acha o maior e o menor inteiro dentro de um vetor de 10 inteiros. O vetor pode ser preenchido de forma aleatória ou informado pelo usuário. Obs: usar apenas as variáveis a seguir:

```
int *vetor, i, *maior, *menor;
```

Exercícios

- Escreva um programa em linguagem C que solicite ao usuário a quantidade de alunos de uma turma e aloca um vetor de notas (números reais). Após a leitura das notas, imprima a média aritmética. Obs: não deve ocorrer desperdício de memória; e após ser utilizada a memória deve ser liberada.

Exercícios

- Desenvolva um programa que calcule a soma de duas matrizes MxN de números inteiros. A implementação deste programa deve considerar as dimensões fornecida pelo usuário (Dica: represente a matriz através de variáveis do tipo int**, usando alocação dinâmica de memória).